

Care Note — Round Two

Response to the sixteen clinic scenarios and the twelve-capability list. Architecture, schema and latency are unchanged and remain in [TECHNICAL BRIEF.md](#); this covers what the review changed. Per-scenario verdicts and their tests are in [SCENARIO_COVERAGE.md](#).

Where we landed: **11 SURVIVES · 4 PARTIAL · 1 DOES NOT** on the scenarios, from 6 · 6 · 4 in our own first assessment. Eleven decisions (D-070 to D-080), 74 new tests, 509 backend and 44 component tests passing.

1. What the review actually found

Our own assessment came out 6/6/4 before any code changed. The pattern in those failures was more useful than any individual fix: **what survived was structural, what failed was operational**. Access control fused into a type, redaction fused into the only code that can reach a network, highlights anchored to a version — all held. What failed was every question of the form *what happens when*: when the provider is down, when the server crashes, when a clinic needs to add a patient on a Tuesday. We had built a record system and had not built the clinic around it.

Two defaults explain most of it, and both are worth stating because they generalise beyond this build.

The stub LLM provider cannot fail. It is in-process, so it cannot time out, refuse a connection or return a 503. Every test run and demo for the entire build executed against a provider physically incapable of failing — which is why a provider outage turned out to be an unhandled 500 with a traceback. The default is still right; it makes the build runnable without an API key. But it bough that determinism by removing the failure surface from view, and nothing made the trade visible. `CARENOTE_LLM_FORCE_UNAVAILABLE` now exercises that path on every run.

The seed script stood in for features. `init_db.py` runs at Phase 1 step 1, so from the first commit every test and demo began with a populated database. *How does a patient come to exist?* never arose, because patients always already did. Anything a seed provides is a feature you have not built and will not notice missing — which is exactly scenario 1.

2. Scenario verdicts

#	Scenario	Was	Now
1	Patient with no email	PARTIAL	SURVIVES — staff enrolment, phone as identifier
2	Clinic isolation, one line	SURVIVES	SURVIVES — <code>AccessScope.query()</code> , three test files
3	Read your logs	DOES NOT	SURVIVES — sanitised middleware; 3 tests fail without it
4	Prove the ordering	SURVIVES	SURVIVES — one exit, asserted by source scan
5	Clinic B on Monday	PARTIAL	PARTIAL — enrolment built; per-clinic config still absent
6	Trilingual consult	PARTIAL	PARTIAL — parity for en/ms; Hokkien flagged, not read
7	Allergy at minute two	DOES NOT	DOES NOT — batch; boundary asserted by test
8	Model hangs 45s	PARTIAL	PARTIAL — 8s timeout and cancel; no server-side abort
9	Provider 503 for an hour	DOES NOT	SURVIVES — degrades to extractive, labelled
10	Two people, same note	SURVIVES	SURVIVES — version check plus unique constraint
11	Link never received	DOES NOT	PARTIAL — reach modelled; no sender exists
12	Summary wrong by one dosage	PARTIAL	SURVIVES — correction banner and a dose gate
13	Allergy asserted vs denied	PARTIAL	SURVIVES — <code>assertion_vs_denial</code> at HIGH
14	A number that means nothing	SURVIVES	SURVIVES — floor now language-independent
15	Ranking learns from what it showed	SURVIVES	SURVIVES — floors, exploration, inspectable
16	Highlight cites edited source	SURVIVES	SURVIVES — now side by side, both versions named

On the twelve capabilities: **6 SURVIVES · 5 PARTIAL · 1 DOES NOT**. The single DOES NOT is streaming ASR. The five partials are speaker diarization (timing arithmetic, not acoustic), code-switching and multilingual downstream (two languages), dosage confirmation (17 drugs, no formulary), and collaborative editing (no lost updates, but not real-time). Full per-row detail with the tests that cover each is in [SCENARIO_COVERAGE.md](#).

3. What changed

Failure handling (3, 8, 9). The chokepoint now translates timeouts, transport errors, 5xx and 429 into one `LLMUnavailableError`; it does not decide what to do about them. Each caller chooses, because the right answer differs by purpose: the scribe degrades to the deterministic extractive summariser it already had — labelled `offline-extractive-v1:provider-unavailable`, so a degraded note is legible as degraded — and a patient-facing generator would refuse. 4xx stays loud: a bad API key hiding behind a slightly worse summary is indistinguishable from an outage. Timeout 60s → 8s, plus a cancel button, since the transcript is stored before the model is called and abandoning loses the summary, never the consult.

Crash logging is a separate chokepoint: type, route and an eight-character reference, never `str(exc)`, which is where `SQLAlchemy` puts bound parameters.

Language parity (6, 14). The risk floor now works over canonical tags rather than English strings, so it inherits every language the tagger knows. It also gained negation handling, asymmetrically: a symptom is dropped only if it appears *exclusively* inside a negation, so "no chest pain Monday, chest pain today" still rates high, and a denied symptom drops to medium rather than to nothing. Content in an unsupported language is flagged as unread rather than silently producing nothing — abstention beats confident silence.

Denials as claims (13). Negated allergy mentions become `allergy_denial` claims instead of being discarded, and `assertion_vs_denial` fires at HIGH — not CRITICAL, because nothing dangerous has happened yet and diluting CRITICAL would break the level that means *someone is about to be given a drug they react to*.

Reach (11, 12). `unread / read / corrected`, derived from timestamps `PatientView` had recorded since D-033 and nothing had queried. `dispatched` is deliberately absent because nothing dispatches. The patient view leads with a plain-language correction banner, computed before the read marker moves.

Enrolment (1, 5). Staff can register a patient and issue a login; identifier type is explicit and a phone number is first-class. `Patient.dob` and `mrn` became nullable — both were `NOT NULL`, which was a second and quieter way the schema decided that someone known only by a phone number was not a patient.

Regeneration (capability list). Was undefined behaviour, found by probing: a new session id produced a duplicate summary, the same one crashed on a unique constraint. Now it reuses the entry and appends a version, so accepted highlights and comments survive — and refuses outright when a human has edited, because merging would mean choosing which of a clinician's sentences to keep.

Dosage (capability list, and the hint). Doses were compared against each other and never against a reference, so the build could see two entries disagree and not see 5000mg. A 17-drug adult reference now bands doses plausible / unusual / implausible; only implausible gates, and only on patient-facing writes, with the override recorded. Acknowledgement rather than refusal: a hard block teaches people to route around the check.

Provenance (16). Stale highlights now show the anchored text and the current text side by side, with both version numbers named.

4. What we tried that did not work

An order-of-magnitude dose threshold. Metformin 5000mg passed as merely unusual — the exact case the hint describes. The fix is principled rather than tuned: ranges are *single-dose*, a legitimate daily total reaches ~3× that (TDS), so beyond 3× exceeds any plausible daily total. 1500mg BD correctly stays unusual.

@app.exception_handler(Exception) for the log leak. Does not work. Starlette's `ServerErrorMiddleware` calls the handler and then *re-raises* so the server can log the traceback — sanitising the response and leaving the leak untouched. It had to be middleware. We only found this because the test asserted on captured log output rather than on the response body.

A blanket-denial pattern that matched too much. `denies allergy to aspirin` registered as a denial of *all* allergies. A contradiction detector that cries wolf is worse than one with gaps: the gap loses a finding, the false positive teaches people to stop reading all of them.

Streaming capture — not attempted. Retrofitting incremental extraction is tractable; `test_capture_timing.py` asserts the deterministic extractors work on two turns alone. What we could not resolve is the interruption policy. When it is acceptable to interrupt a doctor mid-consultation is a clinical workflow decision, and guessing would have produced a demo rather than a product.

5. Assumptions that no longer stand

"Logging is solved because log_event is content-free." Solved for content we log on purpose; silent about content logged on our behalf. One unhandled 500 put a name, an NRIC and note content in a single line. The earlier brief claimed "logs grepped, zero hits" — true of the success path, and the wrong question.

"Multilingual is fixed because the tagger handles Malay" (D-058). The fix landed at the tagger; the risk floor was a separate path making the same English-only assumption. Every Malay fixture passed because they only exercised the tagger. *When you fix an assumption at one layer, go looking for other layers that made the same one.*

"Dropping negated mentions is correct." Correct for its purpose, and it discarded the patient's denial entirely — a denial is a position, not an absence.

"The medication cue vocabulary is adequate." Built against written notes. *I will start you on amoxicillin* — how prescribing is actually said — classified as a bare dose claim and never reached the allergy comparison.

Still standing: regex redaction over NER for auditability (D-012); no HTML-escaping on write, because corrupting dose <5mg is worse than the XSS it prevents (D-015); full snapshots over diffs (D-006); least-privilege on staff visibility (D-004); and reporting contradictions without ever resolving them (D-068) — tested hardest, and the one we would defend most strongly.

6. Where we expect this to fail next

1. **A consult beyond English and Malay.** The unread flag means the system says so, but the content is still invisible downstream. First week.
 2. **A dose the 17-drug table does not know.** No check at all, and the absence looks identical to a pass.
 3. **Alert fatigue on contradictions.** Four classes now fire; nothing monitors the dismissal rate, which is the signature NEVER_DAMPENED exists to survive.
 4. **Regeneration refusing too often.** Any human edit blocks it, including a typo fix — possibly annoying enough to teach people not to edit AI summaries.
 5. **Two disagreeing clinicians in the learning loop.** Untested rather than designed; saturation bounds the damage.
- Still not built, plainly:** streaming ASR, acoustic diarization, any message sender, per-clinic configuration, presence indicators, and a real drug database. Scenario 7 remains DOES NOT and test_capture_timing.py asserts that boundary rather than papering over it.